

Text Processing & Regular Expressions

Lecture 12

Dr. Colin Rundel

Strings

Strings in R and Python

In R a string is a single element of a character vector, and string functions are vectorized over those elements.

In Python a `str` is an immutable sequence of characters that can be indexed and sliced, and its methods act on one string at a time.

```
1 x = c("apple", "banana", "kiwi")
```

R

```
1 nchar(x)
```

R

```
[1] 5 6 4
```

```
1 substr(x, 1, 3)
```

R

```
[1] "app" "ban" "kiw"
```

```
1 toupper(x)
```

R

```
[1] "APPLE" "BANANA" "KIWI"
```

```
1 x = "banana"
```

PY

```
1 len(x)
```

PY

```
6
```

```
1 x[0:3]
```

PY

```
'ban'
```

```
1 x.upper()
```

PY

```
'BANANA'
```

```
1 [  
2     s.upper()  
3     for s in ["apple", "banana", "kiwi"]  
4 ]
```

PY

```
['APPLE', 'BANANA', 'KIWI']
```



stringr

`stringr` is the tidyverse's string package, built on `stringi` and the ICU C library. Base R has equivalents for most of its functions (`paste()`, `substr()`, `nchar()`, `gsub()`, ...), but their names and argument orders are inconsistent.

- Most string-manipulation functions start with `str_`.
- Most take the string as the first argument, making them convenient to use with pipes.
- Functions are vectorized over the string inputs. Some, such as `str_detect()`, also accept a vector of patterns.
- Missing strings generally produce missing results.

String operations

Most everyday string tasks have a direct equivalent in each language.

Task	R / <code>stringr</code>	Python
length	<code>str_length()</code>	<code>len()</code>
substring	<code>str_sub()</code>	<code>s[i:j]</code>
combine	<code>str_c()</code> , <code>str_flatten()</code>	<code>+</code> , <code>sep.join()</code>
literal split	<code>str_split(s, fixed(sep))</code>	<code>s.split(sep)</code>
regex split	<code>str_split(s, pattern)</code>	<code>re.split(pattern, s)</code>
trim whitespace	<code>str_trim()</code> , <code>str_squish()</code>	<code>s.strip()</code> , <code>" ".join(s.split())</code>
pad	<code>str_pad()</code>	<code>s.rjust()</code> , <code>s.ljust()</code> , <code>s.zfill()</code>
case	<code>str_to_upper()</code> , <code>str_to_title()</code>	<code>s.upper()</code> , <code>s.title()</code>
templates	<code>str_glue()</code>	f-strings
number formatting	<code>formatC()</code> , <code>format()</code>	format spec, <code>f"{x:.3f}"</code>
many strings	vectorized	comprehension, <code>.str</code> accessor

Strings in data frames

`stringr` functions are vectorized, so they work directly inside `mutate()`.

Python's string methods are not, so pandas and polars expose vectorized versions through the `.str` accessor.

```
1 d = tibble(  
2   name = c("Bob Smith", "Alice Jones")  
3 )
```

```
1 d |>  
2   mutate(  
3     upper = str_to_upper(name),  
4     n = str_length(name)  
5   )
```

```
# A tibble: 2 × 3  
  name      upper      n  
  <chr>    <chr>    <int>  
1 Bob Smith BOB SMITH      9  
2 Alice Jones ALICE JONES    11
```

```
1 import polars as pl  
2 d = pl.DataFrame({  
3   "name": ["Bob Smith", "Alice Jones"]  
4 })
```

```
1 d.with_columns(  
2   upper = pl.col("name").str.to_uppercase()  
3   n = pl.col("name").str.len_chars()  
4 )
```

shape: (2, 3)

name	upper	n
str	str	u32
"Bob Smith"	"BOB SMITH"	9
"Alice Jones"	"ALICE JONES"	11

The two `.str` accessors are not the same. `pandas` mostly keeps Python's method names (`.str.upper()`, `.str.len()`)

Regular expressions

Regular expressions

A regular expression (regex) is a sequence of characters that describes a pattern to be matched in text. Regexes are used to detect, extract, replace, and split strings, and to validate input.

- The idea dates to the 1950s, when Stephen Kleene formalized regular languages.
- They came into common use in the late 1960s and early 1970s through Unix text processing tools like `ed`, `grep`, and `sed`.
- Most modern languages use a syntax descended from Perl's. The core is shared, but each engine differs in its details.
- In R, `stringr` uses the ICU engine and base R uses `TRE` by default or `PCRE` when `perl = TRUE`.
- We will work through the syntax using `stringr`, and then see how it translates to Python's `re` module.

stringr regular expression functions

Function	Description
<code>str_detect</code>	Detect the presence or absence of a pattern in a string.
<code>str_subset</code>	Subset a vector of strings based on the presence of a pattern.
<code>str_count</code>	Count the number of matches in a string.
<code>str_locate</code>	Locate the first position of a pattern and return a matrix with start and end.
<code>str_extract</code>	Extract the text corresponding to the first match.
<code>str_match</code>	Extract capture groups formed by <code>()</code> from the first match.
<code>str_split</code>	Split a string into pieces and return a list of character vectors.
<code>str_replace</code>	Replace the first matched pattern and return a character vector.
<code>str_remove</code>	Remove the first matched pattern and return a character vector.
<code>str_view</code>	Show the matches made by a pattern.

Many of these functions have variants with an `_all` suffix (e.g. `str_replace_all`) which will match more than one occurrence of the pattern in a string.

Simple pattern detection

```
1 text = c("The quick brown", "fox jumps over", "the lazy dog")
```

R

```
1 str_detect(text, "quick")
```

R

```
[1] TRUE FALSE FALSE
```

```
1 str_subset(text, "o")
```

R

```
[1] "The quick brown" "fox jumps over"  "the lazy dog"
```

```
1 str_detect(text, "the")
```

R

```
[1] FALSE FALSE TRUE
```

```
1 str_detect(text, regex("the", ignore_case = TRUE))
```

R

```
[1] TRUE FALSE TRUE
```

Escape characters

An escape character changes the interpretation of the character(s) that follow it. In R string literals the escape character is `\`.

Literal	Character
<code>\'</code>	single quote
<code>\"</code>	double quote
<code>\\</code>	backslash
<code>\n</code>	new line
<code>\r</code>	carriage return
<code>\t</code>	tab
<code>\b</code>	backspace
<code>\f</code>	form feed

Escapes in practice

`print()` shows a string as it would be typed, escapes included. `cat()` shows the actual characters.

```
1 print("a\tb")
```

```
[1] "a\tb"
```

```
1 cat("a\tb")
```

```
a  b
```

```
1 nchar("a\tb")
```

```
[1] 3
```

```
1 print("a\nb")
```

```
[1] "a\nb"
```

```
1 cat("a\nb")
```

```
a
b
```

```
1 nchar("a\nb")
```

```
[1] 3
```

```
1 print("a\\b")
```

```
[1] "a\\b"
```

```
1 cat("a\\b")
```

```
a\b
```

```
1 nchar("a\\b")
```

```
[1] 3
```

```
1 print("a\"b")
```

```
[1] "a\"b"
```

```
1 cat("a\"b")
```

```
a"b
```

```
1 nchar("a\"b")
```

```
[1] 3
```

Raw strings

A raw string turns off escape processing, so a backslash is just a backslash. As of R 4.0 these are written as `r"(...)"`.

```
1 cat("\\int_0^\\infty 1/e^x")
```

```
\\int_0^\\infty 1/e^x
```

```
1 cat(r"\\int_0^\\infty 1/e^x")
```

```
\\int_0^\\infty 1/e^x
```

```
1 r"\\d+\\.\\d+"
```

```
[1] "\\d+\\.\\d+"
```

```
1 "\\d+\\.\\d+" == r"\\d+\\.\\d+"
```

```
[1] TRUE
```

Regex metacharacters

The power of regular expressions comes from metacharacters, which modify how the pattern is matched.

```
1 . ^ $ * + ? { } [ ] \ | ( )
```

To match one of these literally it needs to be escaped with a `\`. Regex escapes sit on top of string escapes, so without raw strings we need two levels of escaping.

To match	Regex	R literal	R raw
.	\.	"\\."	r"(\.)"
?	\?	"\\?"	r"(\?)"
+	\+	"\\+"	r"(\+)"
\	\\	"\\\\"	r"(\\"")

Matching metacharacters

```
1 str_detect("abc[def", "\\[")
```

R

```
## Error: '\[' is an unrecognized escape
```

```
1 str_detect("abc[def", "\\[")
```

R

```
[1] TRUE
```

```
1 cat("abc\\def")
```

R

```
abc\def
```

```
1 str_detect("abc\\def", "\\\\")
```

R

```
[1] TRUE
```

```
1 str_detect("abc\\def", r"(\)")
```

R

```
[1] TRUE
```

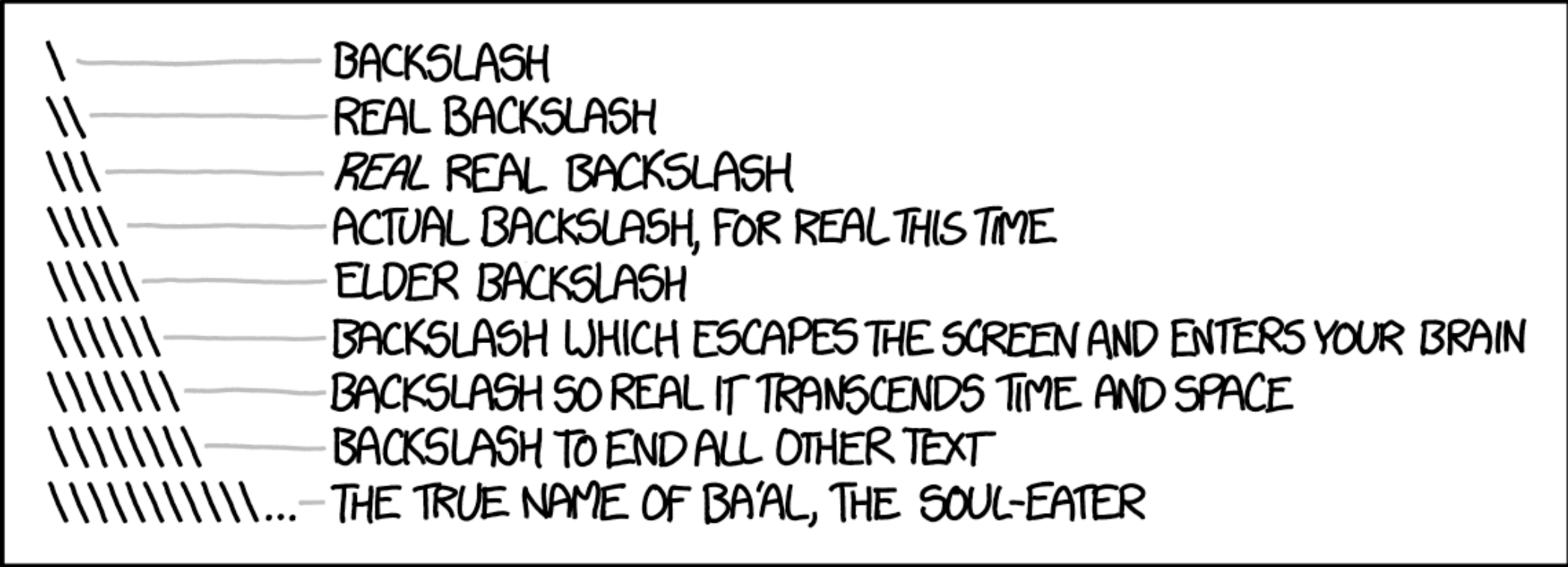
When the pattern is plain text, `fixed()` treats it literally, so regex escaping is unnecessary. Normal R string-literal escaping still applies unless we use a raw string.

```
1 str_detect("abc[def", fixed("["))
```

R

```
[1] TRUE
```

XKCD's take



\ ————— BACKSLASH
\\ ————— REAL BACKSLASH
\\\ ————— REAL REAL BACKSLASH
\\\\ ————— ACTUAL BACKSLASH, FOR REAL THIS TIME
\\\\\\ ————— ELDER BACKSLASH
\\\\\\\\ ————— BACKSLASH WHICH ESCAPES THE SCREEN AND ENTERS YOUR BRAIN
\\\\\\\\\\ ————— BACKSLASH SO REAL IT TRANSCENDS TIME AND SPACE
\\\\\\\\\\\\ ————— BACKSLASH TO END ALL OTHER TEXT
\\\\\\\\\\\\\\\\... — THE TRUE NAME OF BA'AL, THE SOUL-EATER

Anchors

Sometimes we want a pattern to match only at a particular location in a string. Anchors match a position rather than a character.

Regex	Meaning in <code>stringr</code>	Regex	Meaning in <code>stringr</code>
<code>^</code>	Start of string (or line, in multiline mode)	<code>\A</code>	Start of string, always
<code>\$</code>	End of string (or line, in multiline mode)	<code>\Z</code>	End of string or before a trailing newline
<code>\b</code>	Word boundary	<code>\z</code>	End of string, always
<code>\B</code>	Not word boundary		

```
1 text = "the brown owl ate the egg"
```

```
1 str_replace(text, "^the", "---")
```

```
[1] "--- brown owl ate the egg"
```

```
1 str_replace(text, "the$", "---")
```

```
[1] "the brown owl ate the egg"
```

```
1 str_replace(text, "egg$", "---")
```

```
[1] "the brown owl ate the ---"
```

```
1 str_replace_all(text, "e", "*")
```

```
[1] "th* brown owl at* th* *gg"
```

```
1 str_replace_all(text, "e\\b", "*")
```

```
[1] "th* brown owl at* th* egg"
```

```
1 str_replace_all(text, "e\\B", "*")
```

```
[1] "the brown owl ate the *gg"
```

Alternation

If there is more than one pattern we would like to match we can use the or (`|`) metacharacter. It has the lowest precedence of any operator, so use parentheses to limit its scope.

```
1 str_replace_all(text, "the|egg", "---")
```

```
[1] "--- brown owl ate --- ---"
```

```
1 str_replace_all(text, "a|e|i|o|u", "*")
```

```
[1] "th* br*wn *wl *t* th* *gg"
```

```
1 str_replace_all(text, "\\ba|e|i|o|u", "*")
```

```
[1] "th* br*wn *wl *t* th* *gg"
```

```
1 str_replace_all(text, "\\b(a|e|i|o|u)", "*")
```

```
[1] "the brown *wl *te the *gg"
```

Parentheses group alternatives, so `\b(a|e|i|o|u)` applies the word boundary to every vowel. These parentheses also

Classes and ranges

A character class matches any one character from a set, which is written inside square brackets. Ranges simplify classes made up of contiguous letters or digits.

Class	Type
[abc]	Class (a or b or c)
[^abc]	Negated class (not a or b or c)
[a-c]	Range (lower case letter from a to c)
[A-C]	Range (upper case letter from A to C)
[0-7]	Range (digit from 0 to 7)

```
1 text = c("apple", "(219) 733-8965")
```

```
1 str_replace_all(text, "[aeiou]", "*")
```

```
[1] "*ppl*"          "(219) 733-8965"
```

```
1 str_replace_all(text, "[13579]", "*")
```

```
[1] "apple"          "(2**) ***-8*6*"
```

```
1 str_replace_all(text, "[1-5a-f]", "*")
```

```
[1] "*ppl*"          "(**9) 7**-896*"
```

```
1 str_replace_all(  
2   text, "[^1-5a-f]", "*"  
3   )
```

```
[1] "a***e"          "*21*****33*****5"
```

Character classes

Some classes are common enough that they have built-in shorthands.

Meta char	POSIX class	Description
.		Any character except a line terminator, unless <code>dotall = TRUE</code>
<code>\s</code>	<code>[[:space:]]</code>	White space
<code>\S</code>		Not white space
<code>\d</code>	<code>[[:digit:]]</code>	Unicode decimal digit; use <code>[0-9]</code> for ASCII digits
<code>\D</code>		Not digit
<code>\w</code>		Unicode word character, including letters, digits, combining marks, and <code>_</code>
<code>\W</code>		Not word
	<code>[[:alpha:]]</code>	Letter
	<code>[[:punct:]]</code>	Punctuation

POSIX classes like `[[:alpha:]]` belong inside a bracket expression, e.g. `[[:alpha:]]_` or `^[[:space:]]`. `stringr` also

Unicode character classes

`stringr`'s character classes follow Unicode categories, so they include characters beyond ASCII. Here ۳ is the Arabic-Indic digit three, `_` is both a word character and punctuation, and `+` is a symbol rather than punctuation.

```
1 tibble(  
2   char = c("A", "é", "۳", "_", "+"),  
3   digit = str_detect(char, r"(\d)"),  
4   word = str_detect(char, r"(\w)"),  
5   punctuation = str_detect(char, "[[:punct:]]")  
6 )
```

```
# A tibble: 5 × 4  
  char digit word punctuation  
  <chr> <lgl> <lgl> <lgl>  
1 A     FALSE TRUE  FALSE  
2 é     FALSE TRUE  FALSE  
3 ۳     TRUE  TRUE  FALSE  
4 _     FALSE TRUE  TRUE  
5 +     FALSE FALSE FALSE
```

Example

How would we write a regular expression to match a telephone number with the form `(###) ###-####`?

```
1 text = c("apple", "(219) 733-8965", "(329) 293-8753")
```

```
1 str_detect(text, "(\\d\\d\\d) \\d\\d\\d-\\d\\d\\d\\d")
```

```
[1] FALSE FALSE FALSE
```

```
1 str_detect(text, "\\(\\d\\d\\d\\) \\d\\d\\d-\\d\\d\\d\\d")
```

```
[1] FALSE TRUE TRUE
```

Quantifiers

Attached to literals, character classes, or groups, these allow a match to repeat some number of times.

Quantifier	Description
*	Match 0 or more
+	Match 1 or more
?	Match 0 or 1
{3}	Match exactly 3
{3,}	Match 3 or more
{3,5}	Match 3, 4 or 5

Example

How would we improve our previous regular expression for matching a telephone number with the form `(###) ###-####?`

```
1 text = c("apple", "(219) 733-8965", "(329) 293-8753")
```

```
1 str_detect(text, "\\(\\d\\d\\d\\) \\d\\d\\d-\\d\\d\\d\\d")
```

```
[1] FALSE TRUE TRUE
```

```
1 str_detect(text, "\\(\\d{3}\\) \\d{3}-\\d{4}")
```

```
[1] FALSE TRUE TRUE
```

```
1 str_extract(text, "\\(\\d{3}\\) \\d{3}-\\d{4}")
```

```
[1] NA "(219) 733-8965" "(329) 293-8753"
```

Exercise 1

For this exercise, treat the first token as the first name and everything after the first space as the last name, including prefixes and compound names. Match vowels (a, e, i, o, u) without regard to case.

For the following vector of randomly generated names, write a regular expression that,

- detects if the person's first name starts with a vowel (a,e,i,o,u)
- detects if the person's last name starts with a vowel
- detects if either the person's first or last name starts with a vowel
- detects if neither the person's first nor last name starts with a vowel

```
1 x = c("Jeremy Cruz", "Nathaniel Le", "Jasmine Chu", "Bradley Calderon Raygoza",  
2       "Quinten Weller", "Katelien Kanamu-Hauanio", "Zuhriyaa al-Amen",  
3       "Travale York", "Alexis Ahmed", "David Alcocer", "Jairo Martinez",  
4       "Dwone Gallegos", "Amanda Sherwood", "Hadiyya el-Eid", "Shaimaaa al-Can",  
5       "Sarah Love", "Shelby Villano", "Sundus al-Hashmi", "Dyani Loving",  
6       "Shanelle Douglas")
```

R

Extracting matches

`str_extract()` returns the first match in each string, or `NA` when there is none.

`str_extract_all()` returns every match, as a list with one element for each string.

```
1 x = c(  
2   "Midterms on 2026-10-14 and 2026-11-18",  
3   "Final on 2026-12-12",  
4   "No project"  
5 )  
6 date = "\\d{4}-\\d{2}-\\d{2}"
```

```
1 str_extract(x, date)
```

```
[1] "2026-10-14" "2026-12-12" NA
```

```
1 str_extract_all(x, date)
```

```
[[1]]
```

```
[1] "2026-10-14" "2026-11-18"
```

```
[[2]]
```

```
[1] "2026-12-12"
```

```
[[3]]
```

```
character(0)
```

Greedy vs non-greedy matching

At a given starting position, a greedy quantifier tries to consume as much text as possible while allowing the rest of the pattern to match. Adding `?` makes it non-greedy, so it consumes as little as possible while still allowing the rest to match.

```
1 text = "<div class='main'> <div> <a href='here.pdf'>Here!</a> </div> </div>"
```

```
1 str_extract(text, "<div>.*</div>")
```

```
[1] "<div> <a href='here.pdf'>Here!</a> </div> </div>"
```

```
1 str_extract(text, "<div>.*?</div>")
```

```
[1] "<div> <a href='here.pdf'>Here!</a> </div>"
```

Groups

Groups allow you to connect pieces of a regular expression for modification or capture.

Group	Description
<code>(a b)</code>	match literal “a” or “b”, group either
<code>a(bc)?</code>	match “a” or “abc”, group bc or nothing
<code>(abc)def(ghi)</code>	match “abcdefghi”, group abc and ghi
<code>(?:abc)</code>	match “abc”, non-capturing group

Groups in stringr

`str_extract()` returns the full match by default. Its `group` argument can select a capturing group. `str_match()` returns a matrix with the full match in the first column, followed by one column per capturing group.

```
1 text = c("Bob Smith", "Alice Smith", "Apple")
```

```
1 str_extract(text, "^((\\w+) \\w+)")
```

```
[1] "Bob Smith" "Alice Smith" NA
```

```
1 str_extract(text, "^((\\w+) \\w+)", group = 1)
```

```
[1] "Bob" "Alice" NA
```

```
1 str_match(text, "^((\\w+) \\w+)")
```

	[,1]	[,2]
[1,]	"Bob Smith"	"Bob"
[2,]	"Alice Smith"	"Alice"
[3,]	NA	NA

```
1 str_extract(text, "^((\\w+) (\\w+))")
```

```
[1] "Bob Smith" "Alice Smith" NA
```

```
1 str_match(text, "^((\\w+) (\\w+))")
```

	[,1]	[,2]	[,3]
[1,]	"Bob Smith"	"Bob"	"Smith"
[2,]	"Alice Smith"	"Alice"	"Smith"
[3,]	NA	NA	NA

Multiple matches and groups

`str_match_all()` returns a list with one matrix per input string. Each row is a match. The first column contains the full match, followed by one column per capturing group.

```
1 str_match_all("A12 B34", "([A-Z])([0-9]+)")
```

```
[[1]]  
      [,1] [,2] [,3]  
[1,] "A12" "A"  "12"  
[2,] "B34" "B"  "34"
```

Both matches are retained, with the letter and digits captured separately. An input with no matches gets a matrix with zero rows.

Backreferences

Backreferences refer to previously captured groups using `\1`, `\2`, etc. They can be used in a replacement string, or within the pattern itself to match repeated text.

```
1 text = c("Bob Smith", "Alice Smith", "Apple")
```

R

```
1 str_replace(text, "^(\\w+) (\\w+)", "\\2, \\1")
```

R

```
[1] "Smith, Bob" "Smith, Alice" "Apple"
```

```
1 str_detect(c("abab", "cdcd", "abcd"), "^(..)\\1$")
```

R

```
[1] TRUE TRUE FALSE
```

```
1 str_extract(c("hello", "banana", "apple"), "(.)\\1")
```

R

```
[1] "" NA "pp"
```

Flags

Flags change how a pattern is interpreted. In `stringr` they are set by wrapping the pattern in `regex()`.

Argument	Behavior
<code>ignore_case = TRUE</code>	match without regard to case
<code>multiline = TRUE</code>	<code>^</code> and <code>\$</code> match at the start and end of each line
<code>dotall = TRUE</code>	<code>.</code> also matches <code>\n</code>
<code>comments = TRUE</code>	ignore whitespace and <code>#</code> comments in the pattern

```
1 str_count("the cat\nthe hat", "^the")
```

```
[1] 1
```

```
1 str_count("the cat\nthe hat", regex("^the", multiline = TRUE))
```

```
[1] 2
```

Regex in data frames

`stringr`'s functions are vectorized, so they can be used directly on the columns of a data frame.

```
1 d = tibble(x = c("Bob Smith", "Alice Jones"))
```

```
1 d |>
2   mutate(
3     first = str_extract(x, "^\\w+"),
4     s = str_detect(x, "S")
5   )
```

```
# A tibble: 2 × 3
  x           first s
<chr>      <chr> <lgl>
1 Bob Smith   Bob  TRUE
2 Alice Jones Alice FALSE
```

```
1 d |>
2   filter(str_detect(x, "^A"))
```

```
# A tibble: 1 × 1
  x
<chr>
1 Alice Jones
```

Exercise 2

```
1 text = c(  
2   "apple",  
3   "219 733 8965",  
4   "329-293-8753",  
5   "Work: (579) 499-7527; Home: (543) 355 3679"  
6 )
```

R

- Write a regular expression that will extract all of the phone numbers contained in the vector above. Expect 0, 1, 1, and 2 matches across the four inputs.
- Once that works, use capturing groups with `str_match_all()` to retain every phone number while separating the area code from the remaining number. Keep the matches associated with their original input string.

Regular expressions in Python

Python's `re` module

Function	Description
<code>re.search(p, s)</code>	Find the first match anywhere in the string, returns a <code>Match</code> or <code>None</code> .
<code>re.match(p, s)</code>	Match only at the start of the string.
<code>re.fullmatch(p, s)</code>	Match only if the entire string matches.
<code>re.findall(p, s)</code>	Return all matches or captured groups as a list of strings or tuples.
<code>re.finditer(p, s)</code>	Return all matches as an iterator of <code>Match</code> objects.
<code>re.sub(p, repl, s)</code>	Replace all matches with <code>repl</code> .
<code>re.split(p, s)</code>	Split the string at each match.
<code>re.compile(p)</code>	Compile a pattern into an object that can be reused.

Most patterns shown so far translate directly, but regex engines differ in some details as well as their interfaces.

For example `re` does not support the POSIX classes like `[[:digit:]]`. Use `\d`, `\s`, `\w`, or an explicit class such as `[a-z]`.

Raw strings in Python

Python's raw strings are written `r"..."`, and the convention is to write every regex pattern as one.

```
1 print("\\int_0^\\infty 1/e^x")
```

py

```
\int_0^{\infty} 1/e^x
```

```
1 print(r"\\int_0^\\infty 1/e^x")
```

py

```
\int_0^{\infty} 1/e^x
```

```
1 import re
2 text = "the brown owl ate the egg"
```

py

```
1 re.sub(r"\bow", "---", text)
```

py

```
'the brown ---l ate the egg'
```

```
1 re.sub("\\bow", "---", text)
```

py

```
'the brown owl ate the egg'
```

Detecting patterns

`re.search()` takes a single string, so the vectorized `str_detect()` and `str_subset()` become comprehensions.

```
1 text = ["The quick brown", "fox jumps over", "the lazy dog"]
```

py

```
1 [re.search("quick", s) for s in text]
```

py

```
[<re.Match object; span=(4, 9), match='quick'>, None, None]
```

```
1 [bool(re.search("quick", s)) for s in text]
```

py

```
[True, False, False]
```

```
1 [s for s in text if re.search("o", s)]
```

py

```
['The quick brown', 'fox jumps over', 'the lazy dog']
```

```
1 [bool(re.search("the", s)) for s in text]
```

py

```
[False, False, True]
```

```
1 [bool(re.search("the", s, re.IGNORECASE)) for s in text]
```

py

```
[True, False, True]
```

Match objects

`re.search()` returns a `Match` object, or `None` when there is no match. `None` is falsy, so `if m:` is the usual test. Positions are 0-based and the end is exclusive.

```
1 x = "fox jumps over"
```

py

```
1 m = re.search("o", x); m
```

py

```
<re.Match object; span=(1, 2), match='o'>
```

```
1 m.group()
```

py

```
'o'
```

```
1 m.start(), m.end()
```

py

```
(1, 2)
```

```
1 print(re.search("z", x))
```

py

```
None
```

```
1 re.findall("o", x)
```

py

```
['o', 'o']
```

```
1 [  
2     m.start()  
3     for m in re.finditer("o", x)  
4 ]
```

py

```
[1, 10]
```

Replacing and splitting

`re.sub()` plays the role of both `str.replace()` and `str.replace_all()`.
Backreferences in the replacement work the same way as in `stringr`.

```
1 text = "the brown owl ate the egg"
```

py

```
1 re.sub(r"the", "---", text)
```

py

```
'--- brown owl ate --- egg'
```

```
1 re.sub(r"the", "---", text, count=1)
```

py

```
'--- brown owl ate the egg'
```

```
1 re.sub(r"\b(a|e|i|o|u)", "*", text)
```

py

```
'the brown *wl *te the *gg'
```

```
1 re.sub(r"^(\w+) (\w+)", r"\2, \1", "Bob Smith")
```

py

```
'Smith, Bob'
```

```
1 re.split(r"\s+", text)
```

py

```
['the', 'brown', 'owl', 'ate', 'the', 'egg']
```

Match groups in Python

A `Match` object gives access to the capture groups, with `group(0)` being the full match.

```
1 m = re.search(  
2     r"^(\w+) (\w+)", "Bob Smith"  
3 )
```

```
1 m.group(0)
```

```
'Bob Smith'
```

```
1 m.group(1)
```

```
'Bob'
```

```
1 m.group(2)
```

```
'Smith'
```

```
1 m.groups()
```

```
('Bob', 'Smith')
```

Flags and compiled patterns

Flags are passed as an additional argument and combined with `|. re.compile()` stores a pattern and its flags in an object with the same methods as the `re` module.

Behavior	<code>stringr::regex()</code>	Python <code>re</code>
ignore case	<code>ignore_case = TRUE</code>	<code>re.IGNORECASE, re.I</code>
<code>^</code> and <code>\$</code> match each line	<code>multiline = TRUE</code>	<code>re.MULTILINE, re.M</code>
<code>.</code> matches <code>\n</code>	<code>dotall = TRUE</code>	<code>re.DOTALL, re.S</code>
whitespace and comments	<code>comments = TRUE</code>	<code>re.VERBOSE, re.X</code>

```
1 text = "The cat in\nthe hat"
```

```
1 re.findall(r"^the", text)
```

```
[]
```

```
1 re.findall(r"^the", text, re.M)
```

```
['the']
```

```
1 re.findall(r"^the", text, re.M | re.I)
```

```
['The', 'the']
```

```
1 p = re.compile(r"^the", re.M | re.I)
2 p.findall(text)
```

```
['The', 'the']
```

```
1 p.sub("a", text)
```

```
'a cat in\na hat'
```

Regex in pandas and polars

Many `.str` methods in `pandas` and `polars` accept regular expressions. `polars` uses Rust's regex engine, which does not support backreferences in patterns.

```
1 import pandas as pd
2
3 s = pd.Series(
4     ["Bob Smith", "Alice Jones"]
5 )
```

```
1 s.str.extract(r"^(\w+) (\w+)")
```

```
      0      1
0  Bob  Smith
1  Alice Jones
```

```
1 s.str.contains(r"S")
```

```
0    True
1   False
dtype: bool
```

```
1 d = pl.DataFrame({
2     "x": ["Bob Smith", "Alice Jones"]
3 })
```

```
1 d.with_columns(
2     first = pl.col("x").str.extract(r"^(\w+)")
3     s = pl.col("x").str.contains(r"S")
4 )
```

shape: (2, 3)

x	first	s
str	str	bool
"Bob Smith"	"Bob"	true
"Alice Jones"	"Alice"	false

Summary

Regex in R and Python

Task	<code>stringr</code>	Python
pattern string	<code>"\\d+", r"(\d+)"</code>	<code>r"\d+"</code>
detect	<code>str_detect()</code>	<code>re.search()</code>
subset	<code>str_subset()</code>	comprehension with <code>re.search()</code>
first match	<code>str_extract()</code>	<code>m = re.search(p, s)</code> <code>m.group()</code> if <code>m</code> else <code>None</code>
all matches	<code>str_extract_all()</code>	<code>re.findall()</code> , <code>re.finditer()</code>
capture groups	<code>str_match()</code> , <code>str_match_all()</code>	<code>m.group(1)</code> , <code>m.groups()</code> , <code>re.findall()</code>
replace first	<code>str_replace()</code>	<code>re.sub(count=1)</code>
replace all	<code>str_replace_all()</code>	<code>re.sub()</code>
split	<code>str_split()</code>	<code>re.split()</code>
no match: detect	<code>str_detect()</code> → <code>FALSE</code>	<code>re.search()</code> → <code>None</code>
no match: first	<code>str_extract()</code> → <code>NA</code>	check for <code>None</code> before <code>.group()</code>
no match: all	<code>str_extract_all()</code> → <code>character(0)</code> per input	<code>re.findall()</code> → <code>[]</code>
literal pattern	<code>fixed()</code>	<code>in, s.replace(), re.escape()</code>
flags	<code>regex(ignore_case = TRUE)</code>	<code>re.IGNORECASE</code>
data frame columns	vectorized	<code>.str.contains()</code> , <code>.str.extract()</code>

Takeaways

- `stringr` gives R a consistent, vectorized set of string functions. Python's equivalents are methods on a single `str`, applied to many strings with a comprehension or the `.str` accessor of pandas and polars.
- Regular expression syntax is largely shared between the languages. `stringr` generally takes the string first and works over vectors. Python's `re` takes the pattern first and works on one string. Return values depend on the operation.
- Regex escapes sit on top of string escapes. Use raw strings, `r"(...)"` in R and `r"..."` in Python, to avoid doubling every backslash.
- Build patterns up incrementally and test them against examples that should and should not match. If a pattern becomes unreadable, a few simpler steps are usually the better choice.